MERN STACK PRACTICAL CODES

QUESTION 1: Demonstrate use of operators and control statements in JavaScript

Source: Practical 1(e) – operators.js

File: operators.js

```
let x
= 15;
let y
= 10;

console.log("kartvya
pokale");
console.log("Addition:",
x + y);
console.log("Greater
Than:", x > y);

if (x > y) {
console.log("x is
greater than y");
} else {
console.log("y is
greater than x");
}
```

Run: Save as operators.js and run: node operators.js

QUESTION 2: Display a message on the browser using the server response

Setup commands

```
mkdir q2-
server-
message cd
q2-server-
message npm
init -y npm
```

```
install
express
```

File: server.js

```
const express =
require("express
"); const app =
express(); const
PORT = 3000;

app.get("/", (req, res) => {
res.send("<h1>Hello from the Express
Server!</h1><p>Welcome to MERN
Stack.</p>"); });

app.listen(PORT, () => {
console.log(`Server running
at
http://localhost:${PORT}`);
});
```

Run: node server.js, then open http://localhost:3000

QUESTION 3: Write a JavaScript program to print even numbers from 1 to 50

File: even.js

```
console.log("Even numbers from 1
to 50:");

for (let i =
1; i <= 50;
i++) {     if
(i % 2 === 0)
{
console.log(i)
;     }
}
```

Run: Save as even.js and run: node even.js

QUESTION 4: Express fundamentals – create different routes in Express

Setup commands

```
mkdir q4-
express-
routes cd
q4-express-
routes npm
init -y npm
install
express
```

File: server.js

```
const express =
require("express
"); const app =
express(); const
PORT = 3000;

app.get("/", (req,
res) => {
res.send("<h1>Home
Page</h1>"); });

app.get("/about", (req, res) =>
{     res.send("<h1>About
Page</h1><p>This is the About
route.</p>"); });

app.get("/contact", (req,
res) => {
res.send("<h1>Contact
Page</h1><p>Contact us
here.</p>"); });

app.listen(PORT, () => {
console.log(`Server running
at
```

```
http://localhost:${PORT}`);
});
```

Run: node server.js, then visit http://localhost:3000/, /about and /contact

QUESTION 5: Declare variables and write a function to calculate the sum of two numbers and display the result

Source: Practical 3(c) – src/App.jsx

File: src/App.jsx

```
function
App() {
let num1
= 15;
let num2
= 25;


function
add(a, b) {
return a +
b;     }


return
(
<>
            <h1>Practical 3C</h1>
            <h2>Name: kartvya
pokale</h2>
            <h2>First Number:
{num1}</h2>
            <h2>Second Number:
{num2}</h2>
            <h2>Sum = {add(num1,
num2)}</h2>
        </>
```

```
);
}

export default App;
```

Run: Place this in src/App.jsx of a React project, then run: npm run dev

QUESTION 6: Node.js and Express – send a JSON response from the server

Setup commands

```
mkdir q6-
json-
response
cd q6-
json-
response
npm init -
y npm
install
express
```

File: server.js

```
const express =
require("express
"); const app =
express(); const
PORT = 3000;

app.get("/api/stud
ent", (req, res)
=> {
res.json({
name: "Kartvya
Pokale",
course: "BSc IT",
class: "TY BSc
IT",
```

```
rollNo: 17     });
});

app.listen(PORT, () => {
console.log(`Server running
at
http://localhost:${PORT}`);
});
```

Run: node server.js, then open http://localhost:3000/api/student

QUESTION 7: Apply basic CSS styling to a React component

Source: Practical 3(d) – src/App.css and src/App.jsx

File: src/App.css

```
/* App.css */
.container {
text-align:
center;
color: blue;
background-
color:
lightyellow;
padding: 20px;
border-radius:
10px;     font-
family: Arial; }
```

File: src/App.jsx

```
import "./App.css";

function App() {
return (
<div
className="conta
iner">
            <h1>Practical 3D</h1>
            <h2>kartvya
pokale</h2>
```

```
            <h1>Welcome to
React</h1>
            <p>Learning CSS in
React</p>
        </div>

);
}

export default App;
```

Run: Place both files in the React project and run: npm run dev

QUESTION 8: MongoDB – create a database and collection and insert records

Setup commands

```
mkdir q8-
mongodb-
insert cd
q8-mongodb-
insert npm
init -y npm
install
mongodb
```

File: insert.js

```
const { MongoClient } =
require("mongodb");

const client = new
MongoClient("mongodb://127.0.0.1:
27017");

async function
main() {
try {
await
client.connect
();
```

```
        const db =
client.db("CollegeDB");
const students =
db.collection("Students"
);

        const result = await
students.insertMany([
{ name: "Kartvya Pokale", course:
"BSc IT", rollNo: 17 },
            { name: "Anuj",
course: "BSc IT", rollNo: 18 },
            { name: "Yogesh",
course: "BSc IT", rollNo: 19 }
]);


console.log(`${result.insertedCount}
records inserted successfully.`);
} catch (error) {
console.error("MongoDB Error:",
error.message);     } finally {
await client.close();     } }

main();
```

Run: Start MongoDB, then run: node insert.js

Note: MongoDB creates CollegeDB and the Students collection when the records are inserted. Change the URI if your MongoDB server uses another address.

QUESTION 9: Create multiple components and render them on a single page

Source: Practical 4(a) – Header.jsx, Footer.jsx and App.jsx

File: components/Header.jsx

```
function Header() {
return <h1>Welcome to
```

```
MERN Stack
Development</h1>; }

export default Header;
```

File: components/Footer.jsx

```
function Footer() {
return <h3>Thank You
for Visiting</h3>; }

export default Footer;
```

File: src/App.jsx

```
import Header from
"./components/Header";
import Footer from
"./components/Footer";

functio
n App()
{
return
(
<>
            <h1>Practical 4A</h1>
            <h2>kartvya
pokale</h2>
            <Header />
            <p>This is the main
content of the application.</p>
            <Footer />
        </>


);
}

export default App;
```

Run: Create the components folder inside src, save the files, then run: npm run dev

QUESTION 10: Fetch data from the API in a React application

Source: Practical 5(d) – pages/Users.jsx

Setup commands

```
npm install axios
```

File: src/pages/Users.jsx

```
import axios from
"axios"; import {
useEffect, useState }
from "react";

function Users() {
const [users,
setUsers] =
useState([]);

    useEffect(() => {
axios
.get("https://jsonplaceholder
.typicode.com/users")
.then((response) => {
setUsers(response.data);
});     }, []);


return
(
<>
            <h2>User List</h2>
            <ul>
                {users.map((user)
=> (
                    <li
key={user.id}>

{user.name}
                    </li>
```

```
                ))}
            </ul>
        </>

);
}

export default Users;
```

Run: Import and render the Users component from App.jsx, then run: npm run dev

QUESTION 11: React components – pass data between components using props

Source: Practical 4(b) – Student.jsx and App.jsx

File: components/Student.jsx

```
function
Student(pro
ps) {
return (
<>
            <h2>Name:
{props.name}</h2>
            <h3>Course:
{props.course}</h3>
        </>

);
}

export default Student;
```

File: src/App.jsx

```
import Student from
"./components/Student";

functio
n App()
{
```

```
return
(
<>
            <h1>Practical 4B</h1>
            <h2>kartvya
pokale</h2>
            <Student
name="Shadaan Idrisi"
course="MERN Stack" />
        </>

);
}
export default App;
```

Run: Create the components folder inside src, save both files, then run: npm run dev

QUESTION 12: Create an API to add user details using Express and MongoDB

Setup commands

```
mkdir
q12-
user-api
cd q12-
user-api
npm init
-y
npm install express mongodb
```

File: server.js

```
const express =
require("express");
const { MongoClient
} =
require("mongodb");
```

```
const app = express(); const
PORT = 3000; const client =
new
MongoClient("mongodb://127.0
.0.1:27017");

app.use(express.json());

async function
startServer() {
try {         await
client.connect();
console.log("Connecte
d to MongoDB");

        const db =
client.db("UserDB");
const users =
db.collection("Users"
);


app.post("/api/users",
async (req, res) => {
try {
const { name, email, age
} = req.body;

                if (!name ||
!email || age === undefined) {
return res.status(400).json({
                        message:
"Please provide name, email and
age."                     });
}

                const
result = await
users.insertOne({
name,
email,
```

```
age: Number(age)
});


res.status(201).json({
message: "User added
successfully",
userId: result.insertedId
});             } catch (error)
{
res.status(500).json({ message:
error.message });             }
});

        app.get("/", (req, res)
=> res.send("User API is
running."));

        app.listen(PORT, () => {
            console.log(`Server
running at
http://localhost:${PORT}`);
});     } catch (error) {
console.error("Could not start
server:", error.message);     } }

startServer();
```

File: Postman request body (JSON)

```
{
    "name": "Kartvya Pokale",
    "email":
"kartvya@example.com",
    "age": 19
}
```

Run: Start MongoDB and run node server.js. In Postman send POST to http://localhost:3000/api/users; choose Body → raw → JSON.

Note: MongoDB stores the submitted data in the UserDB database and Users collection.

QUESTION 13: Build a To-do list interface using React

Setup commands

```
npm create vite@latest
q13-todo -- --template
react cd q13-todo npm
install
```

File: src/App.jsx

```
import { useState
} from "react";
import
"./App.css";

function App() {
const [task,
setTask] =
useState("");
const [tasks,
setTasks] =
useState([]);

    function addTask() {
if (task.trim() === "") return;
setTasks([...tasks, { id:
Date.now(), text: task.trim()
}]);          setTask("");     }

    function deleteTask(id)
{
setTasks(tasks.filter((item
) => item.id !== id));
}

    return (
<div
className="conta
iner">
            <h1>My
To-do List</h1>
```

```
<input
type="text"
placeholder="Enter a
task"
value={task}

onChange={(e) =>
setTask(e.target.value)}
/>
            <button
onClick={addTask}>Add
Task</button>
            <ul>
                {tasks.map((item)
=> (
                    <li
key={item.id}>

{item.text}
                        <button onClick={() =>
deleteTask(item.id)}>Delete</button>
                    </li>
                ))}
            </ul>
        </div>

);
}

export default App;
```

File: src/App.css

```
.container {
max-width:
500px;
margin: 50px
auto;
padding: 20px;
```

```
text-align:
center;
font-family:
Arial, sans-
serif; } input {
padding: 10px;
width: 60%;
margin-right:
8px; } button {
padding: 10px;
cursor: pointer;
} li {
display: flex;
justify-content:
space-between;
margin: 10px 0;
}
```

Run: Run npm run dev and open the Vite URL shown in the terminal (usually http://localhost:5173).

QUESTION 14: Create different routes in Express

Setup commands

```
mkdir q14-
express-
routes cd
q14-
express-
routes npm
init -y npm
install
express
```

File: server.js

```
const express =
require("express
"); const app =
express(); const
PORT = 3000;
```

```
app.get("/", (req,
res) => {
res.send("<h1>Home
Page</h1>"); });

app.get("/about", (req, res) =>
{     res.send("<h1>About
Page</h1><p>This is the About
route.</p>"); });

app.get("/contact", (req,
res) => {
res.send("<h1>Contact
Page</h1><p>Contact us
here.</p>"); });

app.listen(PORT, () => {
console.log(`Server running
at
http://localhost:${PORT}`);
});
```

Run: node server.js, then visit http://localhost:3000/, /about and /contact

Note: This question has the same wording as Q4 in the supplied question bank; the same Express routing implementation is used.

QUESTION 15: React Routing – create multiple pages and implement navigation between them

Source: Practical 5(b) and 5(c) – Home.jsx, About.jsx, Contact.jsx and App.jsx

Setup commands

```
npm install react-router-dom
```

File: src/pages/Home.jsx

```
function Home() {
return <h2>Welcome
to Home Page</h2>;
}
export default Home;
```

File: src/pages/About.jsx

```
function About() {
return <h2>Welcome
to About Page</h2>;
}
export default About;
```

File: src/pages/Contact.jsx

```
function Contact() {
return <h2>Welcome to
Contact Page</h2>; }
export default Contact;
```

File: src/App.jsx

```
import { BrowserRouter, Routes,
Route, Link } from "react-router-
dom"; import Home from
"./pages/Home"; import About from
"./pages/About"; import Contact
from "./pages/Contact";

function
App() {
return (
```

```
<BrowserRou
ter>
            <nav>
                <Link
to="/">Home</Link> |{" "}
                <Link
to="/about">About</Link> |{" "}
                <Link
to="/contact">Contact</Link>
            </nav>

            <hr />

            <Routes>
                <Route path="/"
element={<Home />} />
                <Route
path="/about" element={<About />}
/>
                <Route
path="/contact" element={<Contact
/>} />
            </Routes>
        </BrowserRouter>

);
}

export default App;
```

Run: Create the pages folder inside src, install react-router-dom, then run: npm run dev

QUESTION 16: Integrate React frontend with Node.js and Express backend

Setup commands

```
mkdir q16-
mern-
integration
cd q16-mern-
integration
```

```
mkdir
backend cd
backend npm
init -y
npm install express cors
```

File: backend/server.js

```
const express =
require("express");
const cors =
require("cors");

const app =
express();
const PORT =
5000;

app.use(cors());
app.use(express.jso
n());

app.get("/api/message",
(req, res) => {
res.json({
message: "Hello from
Node.js and Express!",
student: "Kartvya
Pokale"     }); });

app.listen(PORT, () => {
console.log(`Backend running
at http://localhost:${PORT}`);
});
```

File: Frontend setup (run from q16-mern-integration in a second terminal)

```
npm create vite@latest
frontend -- --template
react cd frontend npm
install
```

File: frontend/src/App.jsx

```
import { useEffect, useState }
from "react";

function App() {
const [data,
setData] =
useState(null);
const [error,
setError] =
useState("");

    useEffect(() => {
fetch("http://localhost:5000/api/me
ssage")
.then((response) => {
if (!response.ok) throw new
Error("Server response error");
return response.json();
})
            .then((result) =>
setData(result))
            .catch((err)
=>
setError(err.message));
}, []);

    return (         <div
style={{ textAlign: "center",
marginTop: "50px" }}>

<h1>React
Frontend</h1>
{error &&
<p>{error}</p>}
            {data ? (
                <>

<h2>{data.message}</h2>
```

```
<p>Student:
{data.student}</p>
                </>
            ) : (
                !error &&
<p>Loading...</p>
            )}
        </div>

);
}

export default App;
```